

```
acct = BankAccount('Alice') # We create an account for Alice.
acct.deposit(120) # _balance can be affected through deposit()
acct.withdraw(40) # _balance can be affected through withdraw()

# Changing _name or _balance outside of BankAccount is impolite, but allowed:
❷ acct._balance = 1000000000
    acct.withdraw(1000)

❸ acct._name = 'Bob' # Now we're modifying Bob's account ledger!
    acct.withdraw(1000) # This withdrawal is recorded in BobLedger.txt!
```

When you run *privateExample.py*, the ledger files it creates are inaccurate because we modified the `_balance` and `_name` outside the class, which resulted in invalid states. *AliceLedger.txt* inexplicably has a lot of money in it:

```
Balance is 0
Deposit 120
Balance is 120
Withdraw 40
Balance is 80
Withdraw 1000
Balance is 999999000
```

Now there's a *BobLedger.txt* file with an inexplicable account balance, even though we never created a `BankAccount` object for Bob:

```
Withdraw 1000
Balance is 999998000
```

Well-designed classes will be mostly self-contained, providing methods to adjust the attributes to valid values. The `_balance` and `_name` attributes are marked as private ❶ ❷, and the only valid way of adjusting the `BankAccount` class's value is through the `deposit()` and `withdraw()` methods. These two methods have checks ❸ ❹ to make sure `_balance` isn't put into an invalid state (such as a negative integer value). These methods also record each transaction to account for the current balance ❺ ❻.

Code outside the class that modifies these attributes, such as `acct._balance = 1000000000` ❷ OR `acct._name = 'Bob'` ❸ instructions, can put the object into an invalid state and introduce bugs (and audits from the bank examiner). By